

Microservices & Gateway — CanBankX

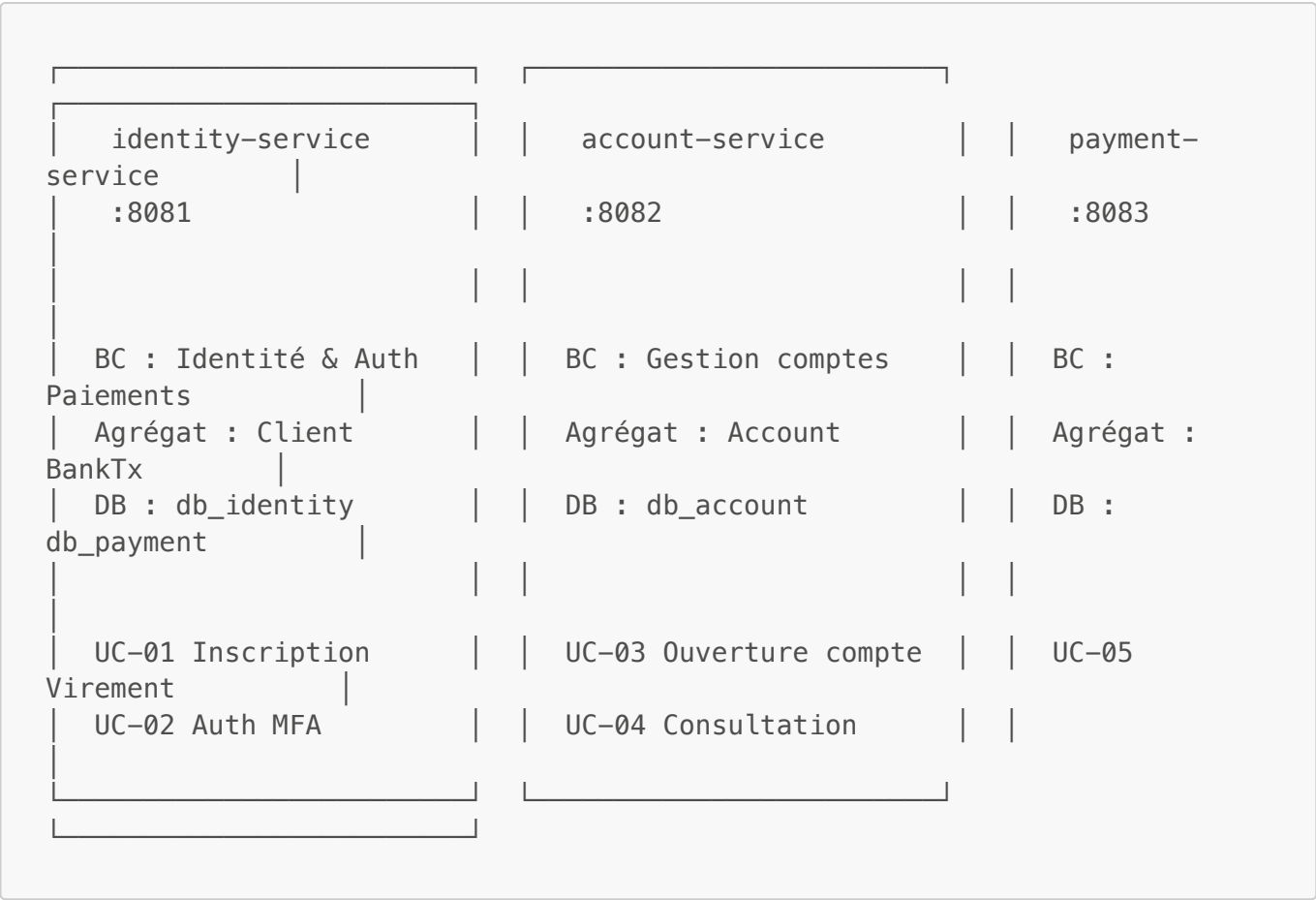
Auteur : Pascal Bourgoin **Date :** Mars 2026 **Compléments :** [arc42.md §4-5](#) · [ADR002-microservices](#) · [ADR004-api](#)

Table des matières

- 1. [Décomposition en Bounded Contexts](#)
- 2. [Module commun `common`](#)
- 3. [KrakenD — API Gateway](#)
- 4. [Load balancing Nginx](#)
- 5. [Communication inter-services](#)
- 6. [Architecture interne de chaque service](#)
- 7. [Profils Docker Compose](#)
- 8. [Limites et dette technique](#)

1. Décomposition en Bounded Contexts

Le système est découpé en **trois microservices**, chacun aligné sur un Bounded Context DDD :



Critères de découpage

Critère	identity	account	payment
Fréquence de changement	Faible	Moyenne	Élevée
Charge attendue	Faible	Moyenne	Très élevée
Scalabilité ciblée	Non requise	Modérée	Critique — répliquable indépendamment
Modèle de données	Profil client, statut, OTP	Solde, type de compte	Transactions, journal d'audit

Principe d'isolation : les services ne s'importent jamais mutuellement comme dépendances Maven. Toute communication cross-service passe exclusivement par HTTP REST. Les références cross-domaine utilisent des identifiants métier (`clientId UUID`, `accountNumber String`) et non des clés étrangères SQL.

2. Module commun `common`

Le module Maven `common` est une bibliothèque partagée importée par les trois services. Il ne contient que des contrats transverses **sans logique métier** :

```
common/src/main/java/com/canbankx/common/
├── exception/
│   ├── ErrorResponse.java           ← DTO de réponse d'erreur (status,
│   │   │   error, message, timestamp)
│   ├── GlobalExceptionHandler.java ← @RestControllerAdvice – intercepte
│   │   │   toutes les exceptions
│   ├── ClientNotFoundException.java
│   ├── AccountNotFoundException.java
│   ├── PaymentNotFoundException.java
│   ├── InsufficientFundsException.java
│   ├── DuplicateEmailException.java
│   └── DuplicateNasException.java
```

Règle d'or : `common` ne dépend d'aucun des trois services. Les services dépendent de `common`. Le graphe de dépendances Maven est acyclique.

```
<!-- dans identity-service/pom.xml, account-service/pom.xml, payment-
service/pom.xml -->
<dependency>
  <groupId>com.canbankx</groupId>
  <artifactId>common</artifactId>
  <version>0.0.1-SNAPSHOT</version>
</dependency>
```

3. KrakenD — API Gateway

KrakenD 2.7 est le point d'entrée unique sur le port **8080**. Sa configuration est entièrement déclarative (JSON) — aucun code Java n'est exécuté dans le chemin de routage.

3.1 Deux fichiers de configuration

Fichier	Mode	Utilisé quand
<code>krakend/krakend-nolb.json</code>	Direct	Mode par défaut (1 instance/service)
<code>krakend/krakend.json</code>	Load balancé	Avec <code>--profile lb</code> (2 instances/service via Nginx)

3.2 Routage par préfixe de chemin

Endpoint Gateway	Backend appelé	Chemin interne
<code>POST /api/clients</code>	identity-service	<code>POST /identityservice/clients</code>
<code>GET /api/clients/{id}</code>	identity-service	<code>GET /identityservice/clients/{id}</code>
<code>PATCH /api/clients/{id}/status</code>	identity-service	<code>PATCH /identityservice/clients/{id}/status</code>
<code>POST /api/auth/login</code>	identity-service	<code>POST /identityservice/auth/login</code>
<code>POST /api/auth/mfa</code>	identity-service	<code>POST /identityservice/auth/mfa</code>
<code>POST /api/accounts</code>	account-service	<code>POST /accountservice/accounts</code>
<code>GET /api/accounts/{id}</code>	account-service	<code>GET /accountservice/accounts/{id}</code>
<code>GET /api/accounts/client/{cid}</code>	account-service	<code>GET /accountservice/accounts/client/{cid}</code>
<code>POST /api/transactions</code>	payment-service	<code>POST /paymentservice/transactions</code>
<code>GET /api/transactions/{id}</code>	payment-service	<code>GET /paymentservice/transactions/{id}</code>
<code>GET /api/transactions/account/{n}</code>	payment-service	<code>GET /paymentservice/transactions/account/{n}</code>

3.3 Agrégation — endpoint `/api/accounts/{id}/summary`

Cet endpoint est le seul à utiliser la **fusion parallèle native de KrakenD** :

```

Client → GET /api/accounts/{id}/summary
        ↳ GET /accountservice/accounts/{id}           (account-
service)
        ↳ GET /paymentservice/transactions/account/{n}/recent
(payment-service)
        ↓ résultats fusionnés
        ← { "account": {...}, "recentTransactions": [...] }

```

Les deux backends sont appelés **en parallèle**. KrakenD attend les deux réponses puis fusionne les objets JSON dans des groupes nommés. Cette agrégation n'existe sous forme de code dans aucun des deux services — elle est entièrement décrite dans `krakend.json`.

3.4 Filtrage des en-têtes

KrakenD ne propage que les en-têtes déclarés dans `"input_headers"` de chaque endpoint :

- `Content-Type`, `Authorization` → tous les backends
- `Idempotency-Key` → uniquement `payment-service` (`POST /api/transactions`)

Tous les autres headers (cookies, `X-Forwarded-*`, headers internes) sont bloqués.

3.5 Timeouts

KrakenD est configuré avec un timeout global de **15 secondes** par backend. Si un service ne répond pas dans ce délai, KrakenD retourne une erreur 500 au client sans rester bloqué indéfiniment.

4. Load balancing Nginx

En mode LB (profil Docker Compose `lb`), **Nginx se place entre KrakenD et les services**. Ce n'est pas Nginx qui reçoit les requêtes des clients — c'est KrakenD qui route vers `nginx-lb`, qui lui-même répartit sur deux instances.

```

Client → KrakenD :8080
        ↳ nginx-lb :8081 → identity-service :8081 (instance
1)
        |
        ↳ identity-service-2 :8081 (instance
2)
        ↳ nginx-lb :8082 → account-service :8082 (instance
1)
        |
        ↳ account-service-2 :8082 (instance
2)
        ↳ nginx-lb :8083 → payment-service :8083 (instance
1)
        |
        ↳ payment-service-2 :8083 (instance
2)

```

4.1 Configuration `nginx/nginx-all.conf`

```
upstream identity_pool {
    least_conn;
    server identity-service:8081;
    server identity-service-2:8081;
}
upstream account_pool {
    least_conn;
    server account-service:8082;
    server account-service-2:8082;
}
upstream payment_pool {
    least_conn;
    server payment-service:8083;
    server payment-service-2:8083;
}

server {
    listen 8081;
    location / { proxy_pass http://identity_pool; }
}
server {
    listen 8082;
    location / { proxy_pass http://account_pool; }
}
server {
    listen 8083;
    location / { proxy_pass http://payment_pool; }
}
```

L'algorithme `least_conn` dirige chaque nouvelle connexion vers l'instance qui possède le **moins de connexions actives** au moment de la décision — plus équitable que round-robin pour des requêtes de durée variable (les transactions sont plus longues que les lectures).

4.2 Récupération sur panne passive

`proxy_next_upstream error timeout` est configuré : si une instance ne répond pas, Nginx retransmet automatiquement la requête à l'autre instance sans que le client remarque la panne.

4.3 Métriques Nginx

En mode LB, le service `nginx-exporter` (port 9113) scrape `/nginx_status` et expose les métriques Nginx au format Prometheus :

- `nginx_connections_active`
- `nginx_http_requests_total`

Ces métriques sont visibles dans le dashboard Grafana.

5. Communication inter-services

Seul `payment-service` initie des appels vers d'autres services. `identity-service` et `account-service` ne font aucun appel sortant.

5.1 Clients HTTP dans payment-service

Deux classes `RestClient` agissent comme **Anti-Corruption Layers** entre `payment-service` et les autres services :

AccountClient — appelle `account-service`

```
PATCH /accountservice/accounts/number/{accountNumber}/debit
Body: { "amount": 100.00 }
→ 200 OK { "id": ..., "balance": 900.00, ... }
→ 422 Unprocessable Entity (si solde insuffisant)
→ 404 Not Found (si compte inexistant)

PATCH /accountservice/accounts/number/{accountNumber}/credit
Body: { "amount": 100.00 }
→ 200 OK { "id": ..., "balance": 1100.00, ... }
```

IdentityClient — appelle `identity-service`

```
GET /identityservice/clients/{clientId}
→ 200 OK { "id": ..., "status": "ACTIVE", ... }
→ 404 Not Found (si client inexistant)
```

5.2 Configuration du pool de connexions

`RestClient` est configuré avec `JdkClientHttpRequestFactory` pour réutiliser les connexions TCP :

```
@Bean
public RestClient accountRestClient() {
    return RestClient.builder()
        .requestFactory(new JdkClientHttpRequestFactory())
        .baseUrl("http://account-service:8082")
        .build();
}
```

Cela évite l'overhead de l'établissement d'une nouvelle connexion TCP à chaque appel inter-service.

5.3 Gestion des erreurs inter-services

`PaymentService` intercepte les codes HTTP de retour d'`AccountClient` et les convertit en exceptions du domaine local :

Réponse account-service	Exception levée	Conséquence
422 au débit	InsufficientFundsException	Transaction → FAILED , AuditLog → DEBIT_FAILED
404 au débit	AccountNotFoundException	Transaction → FAILED
404 au crédit	AccountNotFoundException	Compensation : re-crédit du compte source, transaction → FAILED

6. Architecture interne de chaque service

Chaque service suit l'architecture en couches **Controller → Service → Repository → DB** (ADR001). Aucune hexagonale — le choix a été délibérément pragmatique pour ce contexte académique.

```
src/main/java/com/canbankx/{service}/
├── controller/      ← HTTP, @Valid, mapping DTO ↔ entité
├── service/         ← logique métier, @Transactional, appels inter-services
├── repository/      ← JPA Spring Data (interfaces uniquement)
├── model/           ← @Entity JPA, enums
├── client/          ← RestClient vers d'autres services (payment-service
seulement)
├── dto/             ← Request/Response DTOs (pas d'entités JPA dans les
réponses HTTP)
└── config/         ← SecurityConfig, OpenApiConfig, RestClientConfig
```

Règle d'isolation de couche : les entités **@Entity** ne traversent jamais la couche HTTP. Les controllers reçoivent des ***Request** et retournent des ***Response**. Le mapping est fait dans la couche service ou un ***Mapper** dédié.

7. Profils Docker Compose

```
# Mode direct (défaut)
docker compose up -d

# Mode LB (2 instances + Nginx)
KRAKEND_CONFIG=krakend.json docker compose --profile lb up -d

# Mode LB + tests k6 dans le même réseau Docker
KRAKEND_CONFIG=krakend.json docker compose --profile lb --profile testing
up
```

Profil	Services supplémentaires
--------	--------------------------

Profil	Services supplémentaires
(défaut)	krakend (nolb), identity, account, payment, mysql, redis, mailhog, prometheus, grafana, frontend
lb	+ identity-2, account-2, payment-2, nginx-lb, nginx-exporter
testing	+ k6 (container k6 exécutant <code>load-test.js</code>)

Le fichier `krakend-nolb.json` pointe directement vers les noms de service Docker. Le fichier `krakend.json` pointe vers `nginx-lb` aux ports 8081/8082/8083.

8. Limites et dette technique

Limitation	Impact	Solution en production
Appels inter-services synchrones	Panne d' <code>account-service</code> bloque tout paiement	Message broker (Kafka) + saga asynchrone
Pas de circuit breaker	Cascade de timeouts si un service est lent	Resilience4j avec fallback
Pas de service discovery	Les URLs sont hardcodées dans la config Docker	Consul ou Kubernetes DNS
Pas de JWT	Appels inter-services non authentifiés sur le réseau interne	mTLS ou token de service Keycloak
Pas de transaction distribuée	Cohérence éventuelle avec compensation manuelle	Saga pattern (choreography ou orchestration)